

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置

- 展示
- 展示

- 参考

- CLI 命令

- WebSocket 协议

- REST API
- API 端点目录
- 配置参考

- 环境变量
- 数据库 Schema
- 术语表

- TEMPLATE
 - AGENTS.md Template
 - SOUL.md Template
 - IDENTITY.md Template
 - CAPABILITIES.md Template
 - TOOLS.md Template
 - USER.md Template
 - USER_PREDEFINED.md Template
 - BOOTSTRAP.md Template
 - TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

翻译自 English version

WebSocket 协议

GoClaw gateway WebSocket RPC 接口的协议 v3 规范。

概览

GoClaw 在 `/ws` 暴露 WebSocket 端点。客户端与 gateway 之间的所有通信使用 JSON 帧，共三种类型：`req`（请求）、`res`（响应）和 `event`（服务器推送）。任何连接上的第一个请求必须是 `connect`，用于认证并协商协议版本。

连接 URL: `ws://<host>:<port>/ws`

协议版本: `3`

连接限制

参数	值	说明
读取限制	512 KB	单条消息超过此限制时自动关闭连接
发送缓冲	256 条消息	缓冲满时消息会被丢弃
读取截止时间	60 秒	每条消息或 pong 时重置；超时触发断开
写入截止时间	10 秒	单帧写入超时
Ping 间隔	30 秒	服务器发起的 keepalive ping
速率限制	可配置	gateway config 中的 <code>rate_limit_rpm</code> （0 = 禁用，>0 = 每分钟请求数，burst size 5）

CORS 与 Origin 控制

`allowed_origins` — gateway config 中的字符串数组。为空 = 允许所有 origin（开发模式）。支持 `"*"` 通配符。非浏览器客户端（`Origin` 头为空）始终被允许。

桌面模式 — 设置环境变量 `GOCLAW_DESKTOP=1` 启用宽松 CORS（`Access-Control-Allow-Origin: *`）。附加自定义头：`X-GoClaw-Tenant-Id`、`X-GoClaw-User-Id`。

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

请求帧（req）

由客户端发送以调用 RPC 方法。

```
{
  "type": "req",
  "id": "unique-client-id",
  "method": "chat.send",
  "params": { "message": "Hello", "sessionKey": "user:demo" }
}
```

字段	类型	说明
type	string	始终为 "req"
id	string	客户端生成的唯一 ID，在响应中匹配
method	string	RPC 方法名
params	object	方法参数（可选）

响应帧（res）

由服务器回复请求时发送。

```
{
  "type": "res",
  "id": "unique-client-id",
  "ok": true,
  "payload": { ... }
}
```

错误响应：

```
{
  "type": "res",
  "id": "unique-client-id",
  "ok": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "invalid token",
    "retryable": false
  }
}
```

错误结构：

字段	类型	说明
code	string	机器可读的错误码
message	string	人类可读的描述

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置

- 展示
- 展示

- 参考
- CLI 命令

WebSocket 协议

- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema

术语表

- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

details	any	可选的额外上下文
retryable	boolean	重试是否可能成功
retryAfterMs	integer	建议的重试延迟（毫秒）

事件帧（event）

由服务器主动推送，不需要前置请求。

```
{
  "type": "event",
  "event": "agent",
  "payload": { "type": "chunk", "text": "Hello" },
  "seq": 42,
  "stateVersion": { "presence": 1, "health": 2 }
}
```

字段	类型	说明
type	string	始终为 "event"
event	string	事件名称
payload	any	事件特定数据
seq	integer	单调递增的排序号
stateVersion	object	乐观状态同步的版本计数器（ presence 、 health ）

连接握手

第一个请求必须是 connect 。 gateway 在认证完成前会拒绝其他任何方法。

```
// 请求
{
  "type": "req",
  "id": "init",
  "method": "connect",
  "params": {
    "token": "YOUR_GATEWAY_TOKEN",
    "protocol": 3
  }
}

// 成功响应
{
  "type": "res",
  "id": "init",
  "ok": true,
```

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置
- 展示
- 展示

- 参考
- CLI 命令

WebSocket 协议

- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema

- 术语表
- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

协议版本错误或 token 无效时立即返回 `ok: false`。

user_id 要求： `connect` 中的 `user_id` 参数用于按用户范围隔离会话，为必填项。它是不透明的 VARCHAR(255)。多租户部署时，使用复合格式 `tenant.{tenantId}.user.{userId}` ——GoClaw 使用身份传播并信任上游服务提供正确的身份。

RPC 方法

核心

方法	参数	说明
<code>connect</code>	<code>{token, user_id, sender_id?, locale?}</code>	认证。必须是第一个请求
<code>health</code>	—	Ping / 健康检查
<code>status</code>	—	Gateway 状态
<code>agent</code>	<code>{agentId?}</code>	获取单个 agent 的运行时状态（默认 <code>"default"</code> ）
<code>send</code>	<code>{channel, to, message}</code>	将出站消息路由到外部 channel
<code>providers.models</code>	—	列出所有已配置 LLM provider 的可用模型

聊天

Session 所有权验证（v3）： 全部 5 个 `chat.*` 方法均强制验证 session 所有权。非管理员调用方只能访问自己的 session（通过 `user_id` 匹配）。尝试访问他人 session 返回 `UNAUTHORIZED`。管理员和 gateway-owner 连接跳过此检查。

方法	参数	说明
<code>chat.send</code>	<code>{message, sessionKey?, agentId?}</code>	发送消息；响应通过 <code>agent</code> / <code>chat</code> 事件流式传输
<code>chat.history</code>	<code>{sessionKey}</code>	获取消息历史
<code>chat.abort</code>	<code>{sessionKey}</code>	中止进行中的运行
<code>chat.inject</code>	<code>{sessionKey, content}</code>	注入消息而不触发运行
<code>chat.session.status</code>	<code>{sessionKey}</code>	获取 session 的运行状态和活动阶段

会话

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置

展示

展示

- 参考
- CLI 命令

WebSocket 协议

- REST API
- API 端点目录
- 配置参考
- 环境变量

- 数据库 Schema
- 术语表

- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

<code>sessions.list</code>	<code>{agentId?}</code>	列出会话，可按 agent 过滤
<code>sessions.preview</code>	<code>{sessionKey}</code>	获取会话摘要
<code>sessions.patch</code>	<code>{sessionKey, ...fields}</code>	修改会话元数据
<code>sessions.delete</code>	<code>{key}</code>	删除会话
<code>sessions.reset</code>	<code>{key}</code>	清空会话历史
<code>sessions.compact</code>	<code>{key, keepLast?}</code>	截断历史保留最后 N 条消息（默认 4）；history < 6 时跳过

配置

方法	说明
<code>config.get</code>	获取当前配置（敏感信息已脱敏）
<code>config.apply</code>	完整替换配置
<code>config.patch</code>	修改特定配置字段
<code>config.schema</code>	获取配置的 JSON Schema
<code>config.defaults</code>	获取编译时内置默认值 + agents.defaults overlay（只读，master scope）
<code>config.permissions.list</code>	<code>{agentId, configType?}</code>
<code>config.permissions.check</code>	<code>{agentId, scope, configType, userId}</code>
<code>config.permissions.grant</code>	<code>{agentId, scope, configType, userId, permission, grantedBy?, metadata?}</code>
<code>config.permissions.revoke</code>	<code>{agentId, scope, configType, userId}</code>

Cron

方法	参数	说明
<code>cron.list</code>	<code>{includeDisabled?}</code>	列出 cron 任务
<code>cron.create</code>	cron 任务对象	创建 cron 任务
<code>cron.update</code>	<code>{jobId, ...fields}</code>	更新 cron 任务
<code>cron.delete</code>	<code>{jobId}</code>	删除 cron 任务
<code>cron.toggle</code>	<code>{jobId, enabled}</code>	启用或禁用任务
<code>cron.run</code>	<code>{jobId}</code>	立即触发运行
<code>cron.runs</code>	<code>{jobId}</code>	列出运行历史
<code>cron.status</code>	<code>{jobId}</code>	获取任务状态

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

方法	参数	说明
<code>skills.list</code>	—	列出 skill
<code>skills.get</code>	<code>{id}</code>	获取 skill 详情
<code>skills.update</code>	<code>{id, ...fields}</code>	更新 skill 元数据

Hooks

管理存储在 `agent_hooks` 中的生命周期 hook。完整概念和示例请参阅 [Agent Hooks](#)。

所需角色： list/history 需要 `viewer`；test 需要 `operator`；create/update/delete/toggle 需要 `admin`。

方法	参数	说明
<code>hooks.list</code>	<code>{event?, scope?, agentId?, enabled?}</code>	列出调用者 scope 内可见的 hook
<code>hooks.create</code>	hook config 对象	创建 hook；返回 <code>{hookId}</code>
<code>hooks.update</code>	<code>{hookId, updates}</code>	修改 hook 字段；合并后重新验证配置
<code>hooks.delete</code>	<code>{hookId}</code>	删除 hook（builtin hook 返回错误）
<code>hooks.toggle</code>	<code>{hookId, enabled}</code>	启用或禁用 hook
<code>hooks.test</code>	<code>{config, sampleEvent?}</code>	Dry-run hook 配置；不写入 audit 行
<code>hooks.history</code>	—	列出 <code>hook_executions</code> 审计记录

`hooks.list` — 过滤参数：

参数	类型	说明
<code>event</code>	string	按事件名过滤（如 <code>pre_tool_use</code> ）
<code>scope</code>	string	按 scope 过滤： <code>global</code> 、 <code>tenant</code> 、 <code>agent</code>
<code>agentId</code>	string (UUID)	过滤到特定 agent
<code>enabled</code>	boolean	按启用状态过滤

`hooks.create` — 请求参数（所有字段遵循 `HookConfig` schema）：

字段	类型	必填	说明
<code>event</code>	string	是	生命周期事件名称
<code>handler_type</code>	string	是	<code>command</code> 、 <code>http</code> 或 <code>prompt</code>

G

GoClaw Docs

🔍

Home

GitHub

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

所有文档可用格式

字段

类型

必填

说明

scope	string	是	global、tenant 或 agent
name	string	否	人类可读标签
matcher	string	否	tool name 正则
if_expr	string	否	CEL 表达式（替代 matcher）
timeout_ms	int	否	每 hook 超时 ms（默认 5000，最大 10000）
on_timeout	string	否	block（默认）或 allow
priority	int	否	越高越先运行
enabled	bool	否	默认 true
config	object	是	handler 特定子配置
agent_ids	array	否	scope=agent 时的 UUID 列表

hooks.test 响应:

```
{  "result": {    "decision": "allow",    "reason": "...",    "durationMs": 42,    "stdout": "...",    "stderr": "...",    "statusCode": 200,    "updatedInput": {}  }}
```

Channel

方法	说明
channels.list	列出活跃 channel
channels.status	获取 channel 健康状态
channels.toggle	启用/禁用 channel
channels.instances.list	列出数据库中的 channel 实例
channels.instances.get	获取 channel 实例
channels.instances.create	创建 channel 实例
channels.instances.update	更新 channel 实例
channels.instances.delete	删除 channel 实例

配对

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

所有文档可用格式

device.pair.request

{channel, chatId}

请求配对码

device.pair.approve

{code, approvedBy}

批准配对请求

device.pair.deny

{code}

拒绝配对请求

device.pair.list

—

列出待处理和已批准的配对

device.pair.revoke

{channel, senderId}

撤销配对

执行审批

方法	说明
exec.approval.list	列出待处理的 shell 命令审批
exec.approval.approve	批准命令
exec.approval.deny	拒绝命令

团队

方法	说明
teams.list	列出所有团队
teams.create	创建团队（仅管理员）
teams.get	获取团队及其成员
teams.update	更新团队属性
teams.delete	删除团队
teams.members.add	向团队添加 agent
teams.members.remove	从团队移除 agent
teams.tasks.list	列出团队任务（可过滤）
teams.tasks.get	获取任务及其评论/事件
teams.tasks.create	创建任务
teams.tasks.claim	认领任务（标记为进行中）
teams.tasks.assign	将任务分配给成员
teams.tasks.approve	批准已完成的任务
teams.tasks.reject	拒绝任务提交
teams.tasks.comment	向任务添加评论
teams.tasks.comments	列出任务评论
teams.tasks.events	列出任务事件历史

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置
- 展示
- 展示

参考

CLI 命令

WebSocket 协议

- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema

术语表

- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

<code>teams.tasks.delete</code>	删除任务
<code>teams.tasks.active-by-session</code>	获取会话的活跃任务（用于会话切换时恢复状态）
<code>teams.workspace.list</code>	列出团队工作区文件
<code>teams.workspace.read</code>	读取工作区文件
<code>teams.workspace.delete</code>	删除工作区文件
<code>teams.events.list</code>	列出团队事件历史（分页）
<code>teams.known_users</code>	获取团队中已知用户 ID
<code>teams.scopes</code>	获取任务路由的 channel/chat 范围

用量与配额

方法	说明
<code>usage.get</code>	Token 用量统计
<code>usage.summary</code>	用量摘要卡片
<code>quota.usage</code>	当前用户的配额消耗

日志

方法	参数	说明
<code>logs.tail</code>	<code>{action: "start" "stop", level?}</code>	启动或停止实时日志流；活跃时日志条目通过服务器推送事件到达

心跳（Heartbeat）

方法	参数	说明
<code>heartbeat.get</code>	<code>{agentId}</code>	获取 agent 的心跳配置
<code>heartbeat.set</code>	<code>{agentId, enabled?, intervalSec?, prompt?, providerName?, model?, ...}</code>	创建或更新心跳配置（intervalSec 最小 300）
<code>heartbeat.toggle</code>	<code>{agentId, enabled}</code>	启用或禁用心跳
<code>heartbeat.test</code>	<code>{agentId}</code>	立即触发一次心跳运行
<code>heartbeat.logs</code>	<code>{agentId, limit?, offset?}</code>	列出心跳执行日志

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

heartbeat.checklist.get

{agentId}

读取
HEARTBEAT.md 上
下文文件

heartbeat.checklist.set

{agentId, content}

写入/替换
HEARTBEAT.md 上
下文文件

heartbeat.targets

{agentId}

列出心跳通知的投
递目标

API Keys

方法	参数	说明
api_keys.list	—	列出 API key（非管 理员仅见自己的）
api_keys.create	{name, scopes, expires_in?, owner_id?, tenant_id?}	创建 API key；仅返 回一次原始 key
api_keys.revoke	{id}	撤销 API key（非管 理员只能撤销自己 的）

语音（Voices / TTS）

方法	参数	说明
voices.list	—	列出当前租户的 ElevenLabs voices（带缓存）
voices.refresh	—	失效缓存并从 provider 重新拉取 voices

Workstations

仅 Standard 版本。 Lite 版 gateway 不注册这些方法；调用返回
method_not_found。每个方法均需 admin 角色。

方法	参数	说明
workstations.list	—	列出当前租户的 workstation
workstations.get	{id}	获取 workstation 详情
workstations.create	{workstationKey, name, backendType, metadata, defaultCwd?, defaultEnv?}	创建 workstation （ backendType 为 ssh 或 docker ）
workstations.update	{id, ...fields}	更新 workstation

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置

- 展示
- 展示
- 参考
- CLI 命令

WebSocket 协议

- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema
- 术语表
- TEMPLATE
 - AGENTS.md Template
 - SOUL.md Template
 - IDENTITY.md Template
 - CAPABILITIES.md Template
 - TOOLS.md Template
 - USER.md Template
 - USER_PREDEFINED.md Template
 - BOOTSTRAP.md Template
 - TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

方法	参数	说明
<code>workstations.delete</code>	<code>{id}</code>	删除 workstation
<code>workstations.testConnection</code>	<code>{id}</code>	探测 backend 连通性，不执行命令
<code>workstations.linkAgent</code>	<code>{workstationId, agentId, isDefault?}</code>	将 agent 链接到 workstation
<code>workstations.unlinkAgent</code>	<code>{workstationId, agentId}</code>	移除链接
<code>workstations.permissions.list</code>	<code>{workstationId}</code>	列出白名单模式
<code>workstations.permissions.add</code>	<code>{workstationId, pattern, description?, enabled?}</code>	追加白名单模式
<code>workstations.permissions.remove</code>	<code>{workstationId, permissionId}</code>	删除某个模式
<code>workstations.permissions.toggle</code>	<code>{workstationId, permissionId, enabled}</code>	启用/禁用某个模式
<code>workstations.activity.list</code>	<code>{workstationId, limit?, before?, action?}</code>	分页活动审计日志； <code>action</code> 过滤接受 <code>exec</code> 或 <code>deny</code>

租户（Tenants）

方法	参数	说明
<code>tenants.list</code>	—	列出所有租户（仅 owner）
<code>tenants.get</code>	<code>{id}</code>	按 ID 获取租户
<code>tenants.create</code>	<code>{name, slug, settings?}</code>	创建租户及其工作区
<code>tenants.update</code>	<code>{id, name?, status?, settings?}</code>	更新租户属性
<code>tenants.users.list</code>	<code>{tenant_id}</code>	列出租户中的用户
<code>tenants.users.add</code>	<code>{tenant_id, user_id, role?}</code>	添加用户（角色：owner/admin/operator/member/viewer）
<code>tenants.users.remove</code>	<code>{tenant_id, user_id}</code>	移除用户并广播 access-revoked 事件
<code>tenants.mine</code>	—	获取当前用户的租户成员关系

G

GoClaw Docs

Q

消息 (Messaging)

Home

GitHub

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

所有文档可用格式

方法	参数	说明
whatsapp.qr.start	{instance_id}	启动 WhatsApp QR 登录流程
zalo.personal.qr.start	{instance_id}	启动 Zalo Personal QR 登录流程
zalo.personal.contacts	{instance_id}	获取 Zalo 好友和群组

状态：已规划 — whatsapp.qr.start 、 zalo.personal.qr.start 和 zalo.personal.contacts 的协议常量已定义，但 gateway 中对应的处理器尚未实现。

Bitrix Portals

对 Bitrix24 门户连接的自助管理。所有方法都按租户限定范围（从连接解析，绝不从调用方参数获取）。list 和 get_install_url 对任何已认证的租户成员开放；create 和 delete 需要管理员角色。任何响应中都不会返回凭证。

方法	参数	说明
bitrix.portals.list	—	列出当前租户的门户（凭证已脱敏）
bitrix.portals.create	{name, domain, client_id, client_secret}	创建门户；返回 {name, domain, install_url}
bitrix.portals.get_install_url	{name}	为已有门户重新生成安装 URL → {install_url}
bitrix.portals.delete	{name}	删除门户 → {status: "deleted"}

bitrix.portals.create 参数：

参数	类型	说明
name	string	门户 slug — 小写字母、数字、连字符、下划线（2–64 字符）
domain	string	门户主机 — *.bitrix24.{com,eu,ru,...} 或 *.bitrix.info
client_id	string	Bitrix OAuth client ID
client_secret	string	Bitrix OAuth client secret

若 gateway 尚未观测到其公网 URL，create 会以 FAILED_PRECONDITION 失败 — 请先通过 gateway 的外部 URL 打开 UI，以便生成安装 URL。当仍有 channel instance 引用该门户时，delete 会被阻止（FAILED_PRECONDITION）。

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置

展示

展示

- 参考
- CLI 命令
- WebSocket 协议

- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema
- 术语表
- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)

- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题

服务器推送事件

Agent 事件 ("agent")

在 agent 运行期间发出。检查 `payload.type` :

<code>payload.type</code>	说明
<code>run.started</code>	Agent 运行开始
<code>run.completed</code>	运行成功完成
<code>run.failed</code>	运行遇到错误
<code>run.cancelled</code>	运行在完成前被取消
<code>run.retrying</code>	运行正在重试
<code>tool.call</code>	工具被调用
<code>tool.result</code>	工具返回结果
<code>block.reply</code>	回复被输入 guard 拦截
<code>activity</code>	Agent 活动更新

聊天事件 ("chat")

<code>payload.type</code>	说明
<code>chunk</code>	流式文本 token
<code>message</code>	完整消息 (非流式)
<code>thinking</code>	扩展思考 / 推理输出

系统及其他事件

事件	说明
<code>health</code>	定期 gateway 健康 ping
<code>tick</code>	心跳 tick
<code>shutdown</code>	Gateway 正在关闭
<code>cron</code>	Cron 任务状态变更
<code>exec.approval.requested</code>	Shell 命令需要用户审批
<code>exec.approval.resolved</code>	审批决定已做出
<code>device.pair.requested</code>	来自 channel 用户的新配对请求
<code>device.pair.resolved</code>	配对已批准或拒绝
<code>presence</code>	用户在线状态变更

- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置
- 展示
- 展示
- 参考
- CLI 命令
- WebSocket 协议
- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema
- 术语表
- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)
- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题
- 所有文档可用格式

agent.summoning	Predefined agent persona 生成中
delegation.started	委派子 agent 开始
delegation.completed	委派成功完成
delegation.failed	委派失败
delegation.cancelled	委派被取消
delegation.progress	委派的中间结果
delegation.announce	批量子 agent 结果送达父 agent
delegation.accumulated	累积的委派结果
connect.challenge	已发出身份验证挑战
voicewake.changed	语音唤醒词设置已更改
talk.mode	对话模式状态变更
node.pair.requested	收到节点配对请求
node.pair.resolved	节点配对已解决
session.updated	聊天会话元数据已更新
trace.updated	Agent trace 已更新
heartbeat	心跳执行事件
workspace.file.changed	团队工作区文件已更改
agent_link.created	委派链接已创建
agent_link.updated	委派链接已更新
agent_link.deleted	委派链接已删除
tenant.access.revoked	用户的租户访问权限已撤销
zalo.personal.qr.code	Zalo QR 码已生成
zalo.personal.qr.done	Zalo QR 登录已完成

Skill 事件

事件	说明
skill.deps.checked	Skill 依赖检查已开始
skill.deps.complete	所有 skill 依赖已解决
skill.deps.installing	Skill 依赖安装已开始
skill.deps.installed	Skill 依赖安装已完成
skill.dep.item.installing	单个依赖项正在安装
skill.dep.item.installed	单个依赖项安装完成

升级

实战方案

个人助手

团队聊天机器人

客户支持

代码审查 Agent

多频道设置

展示

展示

参考

CLI 命令

WebSocket 协议

REST API

API 端点目录

配置参考

环境变量

数据库 Schema

术语表

TEMPLATE

AGENTS.md Template

SOUL.md Template

IDENTITY.md Template

CAPABILITIES.md Template

TOOLS.md Template

USER.md Template

USER_PREDEFINED.md Template

BOOTSTRAP.md Template

TEAM.md (系统生成)

故障排除

常见问题

Channel 问题

Provider 问题

WebSocket 问题

MCP 问题

数据库问题

Agent Team 问题

所有文档可用格式

事件	说明
team.created	团队已创建
team.updated	团队已更新
team.deleted	团队已删除
team.member.added	成员已加入团队
team.member.removed	成员已从团队移除
team.message.sent	团队内点对点消息
team.leader.processing	团队 leader 正在处理请求
team.task.created	任务已创建
team.task.completed	任务已完成
team.task.claimed	任务已被认领
team.task.cancelled	任务已取消
team.task.failed	任务失败
team.task.reviewed	任务已审核
team.task.approved	任务已批准
team.task.rejected	任务已拒绝
team.task.progress	任务进度更新
team.task.commented	任务已添加评论
team.task.assigned	任务已分配给成员
team.task.dispatched	任务已分发
team.task.updated	任务已更新
team.task.deleted	任务已删除
team.task.stale	任务标记为过期
team.task.attachment_added	任务已添加附件

示例会话

```
const ws = new WebSocket("ws://localhost:18790/ws");

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: "req", id: "1", method: "connect",
    params: { token: "YOUR_TOKEN", user_id: "user-123", protocol: "websocket" }
  }));
};
```


- 升级
- 实战方案
- 个人助手
- 团队聊天机器人
- 客户支持
- 代码审查 Agent
- 多频道设置
- 展示
- 展示
- 参考
- CLI 命令
- WebSocket 协议
- REST API
- API 端点目录
- 配置参考
- 环境变量
- 数据库 Schema
- 术语表
- TEMPLATE
- AGENTS.md Template
- SOUL.md Template
- IDENTITY.md Template
- CAPABILITIES.md Template
- TOOLS.md Template
- USER.md Template
- USER_PREDEFINED.md Template
- BOOTSTRAP.md Template
- TEAM.md (系统生成)
- 故障排除
- 常见问题
- Channel 问题
- Provider 问题
- WebSocket 问题
- MCP 问题
- 数据库问题
- Agent Team 问题
- 所有文档可用格式

```
};

ws.onmessage = (e) => {

  const frame = JSON.parse(e.data);

  // connect 成功后发送聊天消息
  if (frame.type === "res" && frame.id === "1" && frame.ok) {
    ws.send(JSON.stringify({
      type: "req", id: "2", method: "chat.send",
      params: { message: "Hello!", sessionId: "user:demo" }
    }));
  }

  // 流式接收响应 token
  if (frame.type === "event" && frame.event === "chat") {
    if (frame.payload?.type === "chunk") {
      process.stdout.write(frame.payload.text ?? "");
    }
  }
};
```

下一步

- REST API — agent CRUD、skill 上传、traces 的 HTTP 端点
- CLI 命令 — 从终端进行配对和会话管理
- 词汇表 — Session、Lane、Compaction 等核心术语

GitHub

d85bf171

最后更新于 2026-06-07